

SPM: Structured Parallel Programming

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Introduction and concepts
- Stream parallel patterns: pipeline, and farm
- Data parallel patterns: map, reduce, stencil, D&C
- Cost model and execution time

What is and main motivations

- **Structured parallel programming** is a methodology that aims to simplify the development of parallel applications by composing them from a small set of well-defined recurring patterns
 - It emphasizes clarity, maintainability, and efficiency over low-level, ad hoc coding
- The structured approach contrasts with traditional methods, which mainly target absolute efficiency and often lead to more complex, error-prone, and difficult-to-maintain code
- Motivations:
 - There is a limited number of recurring parallelization structures that can be recognized at a proper abstraction level (e.g., pipeline, task farm, partitioning)
 - These recurring “forms of parallelism” have established, reusable implementations that can be “compiled” into the underlying middleware.
 - It’s easier to parallelize a given problem by thinking about how to reduce the problem to a combination of the known “forms” of parallelism

Structured vs. Unstructured programming

- **Structured parallel programming**
 - Built on a pre-defined set of well-assessed, composable parallel forms
 - Emphasizes maintainability and a clear hierarchical task structure
 - Encapsulates low-level details within the middleware/runtime layers
- **Unstructured parallel programming**
 - Involves ad hoc concurrency using low-level mechanisms (explicit thread creation, locks, explicit messaging)
 - Mixes business logic code with parallel coordination synchronization code
 - Offers great flexibility and fine-grained control at the expense of higher complexity and error-proneness
 - Forces developers to manage coordination, synchronization, and data consistency manually
- Structured parallel programming emphasizes the **separation of concerns design principles**

Terminology

- **Parallel patterns:** Recurring, high-level abstractions that describe common ways to organize parallel computation
 - They are reusable, high-level **design templates** that capture common structures in parallel programs (e.g., pipeline, task farm, scan, stencil, map/reduce) that can be applied to solve classes of parallel problems
- **Algorithmic skeletons:** **Concrete implementations** or library constructs that encapsulate specific parallel patterns (or combinations of patterns) into **ready-to-use parallel components**
 - They provide a structured way to express parallelism and often come with pre-optimized implementations for specific hardware architectures. Skeletons are typically provided by libraries or frameworks to simplify parallel programming
- **Parallel paradigms:** Broad conceptual frameworks or models for organizing and thinking about parallel computation.
 - They provide an overall classification of parallelism approaches, incorporating both the structured and unstructured methodologies. Parallel paradigms are often tied to specific programming models (shared-memory, distributed-memory, dataflow, BSP, Actor model)

Objectives of high-level abstractions

- **Hide low-level complexity**
 - Relieve the programmers from implementing the mechanisms and policies needed to implement the parallel pattern
 - Prevent repetitive boilerplate code, letting programmers focus on application logic
- **Enforce the separation of concerns design principles**
 - Keep domain-specific functionality distinct from concurrency management
 - Increase maintainability
- **Promote reusability**
 - Provide standard parallel patterns that can be applied across various parallel applications
- Examples of widely used parallel patterns:
 - **MapReduce**: splits data into chunks (map), processes them in parallel, and merges results (reduce)
 - **Pipeline**: splits the computation into a set of stages, each working simultaneously on different steps of the data flow
 - **Master-Worker**: a central coordinator (master) distributes tasks among multiple Workers that execute them in parallel

Basic Performance Metrics (recap)

- **Key definitions for system performance**

- **Latency (L):** is the time from the moment an input (request) arrives until the output (response) is ready to be delivered.
 - In the queueing theory and networking contexts, Latency is also known as **Response Time**. It includes any waiting time plus the service time
- **Completion time (T_c):** is the total time an application takes to complete all tasks, from start to delivery of all results
 - $L = T_c$ if there is only one task to compute for the application
 - Often referred to as the **makespan**
- **Service time (T_s):** the time required by the system to process a task, excluding waiting time
 - At steady state and full utilization, the time between two consecutive task completions approximates the service time.
- **Throughput :** it is the number of tasks completed per unit of time
 - At steady state and full utilization, it is the inverse of the service time ($\frac{1}{T_s}$);
- **Inter-arrival time (T_a):** the average time between two consecutive task arrivals at the system

Stream parallelism (recap)

- A **stream of tasks** refers to a continuous, **ordered** sequence of computational or operational units (called **tasks**) of the same type that are processed one after the other:

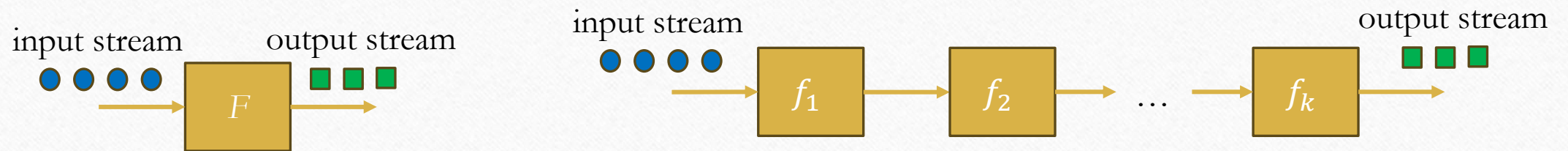
$$\|x_1, x_2, \dots, x_m, \dots\| \text{ where } x_1 < x_2 < \dots < x_m < \dots$$

- Each task in the stream typically represents a discrete piece of work, such as a computation, data transformation, or process operation (e.g., data transfer or coordination), which contributes to the overall goal of the system or application
- In streaming computation, usually but not always, the stream length is **infinite** or very large
 - Terminology: stream **source** and **sink** are the first/last stages of the computation workflow (usually a **pipeline**) that produce and consume the stream of tasks, respectively
- The main parallel pattern dealing with stream parallelism is the **pipeline**
- Objective: **reduce the service-time**

Pipeline

1/2

- **Pipeline parallelism** is a parallel design pattern that enhances computational efficiency by dividing a computation into a sequence of stages, where each stage processes data and passes its output to the next stage. The stages operate concurrently, enabling overlapping computation and improved throughput
 - This method is analogous to an automobile assembly line, where each station performs a specific task on different units simultaneously, leading to the completion of a product
- Objective: **improving throughput** by overlapping the execution of distinct input tasks



$$\|x_1, x_2, \dots, x_m, \dots\|, \quad F(x_m) = f_k(f_{k-1}(\dots f_1(x_m) \dots))$$

Pipeline

2/2

- A bit more formally:

a pipeline computes in parallel the composition of several functions $f_1, \dots, f_k$ over all items of an input stream $\|x_1, x_2, \dots\|$ such that:

$$\begin{aligned} F &= f_k \circ f_{k-1} \circ \dots \circ f_1 \\ \text{pipe}(f_1, \dots, f_k)(\|x_1, x_2, \dots\|) &= \\ \|F(x_1), F(x_2), \dots\| &= \\ \|f_k(f_{k-1}(\dots f_1(x_1) \dots)), f_k(f_{k-1}(\dots f_1(x_2) \dots)), \dots\| \end{aligned}$$

The input is a stream of elements of type X , i.e., $\|x_1, x_2, \dots\|, x_i \in X$, so

$$f_1: X \rightarrow Y_1, f_2: Y_1 \rightarrow Y_2, f_3: Y_2 \rightarrow Y_3, \dots, f_k: Y_{k-1} \rightarrow Y$$

thus, the type of the pipeline is:

$$\text{pipe}(f_1, \dots, f_k): \|X\| \rightarrow \|Y\|$$

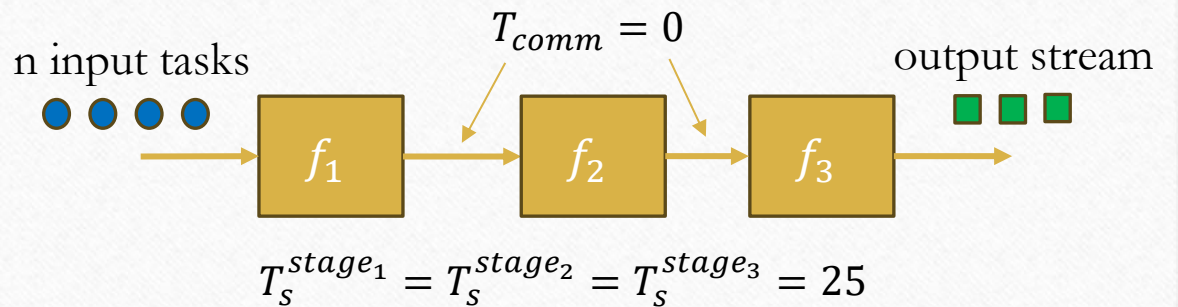
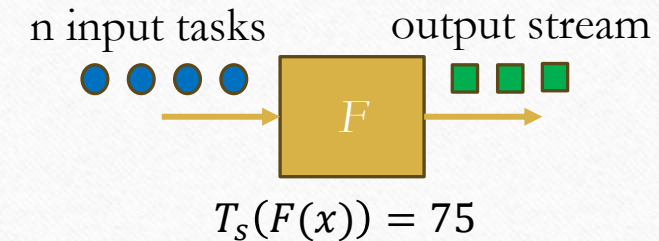
Why is it efficient?

1/2

- **Why does pipeline parallelism increase efficiency?**
 - Because of the overlapping of computations of f_i on different input tasks

- Example, suppose that:

- $F(x)$ takes 75 time units for each input x , i.e., $T_s^{seq} = 75$
- $F()$ can be split in 3 sub-functions $F(x) = f_3(f_2(f_1(x)))$;
- $f_i()$ takes about 25 time units (i.e., $T_s^{stage} = 25$)
- n input tasks to compute
- cost of inter-stage communication 0 (or negligible)
- What is the **throughput** of the 3-stage pipeline?
- What is the **service time** of the 3-stage pipeline?

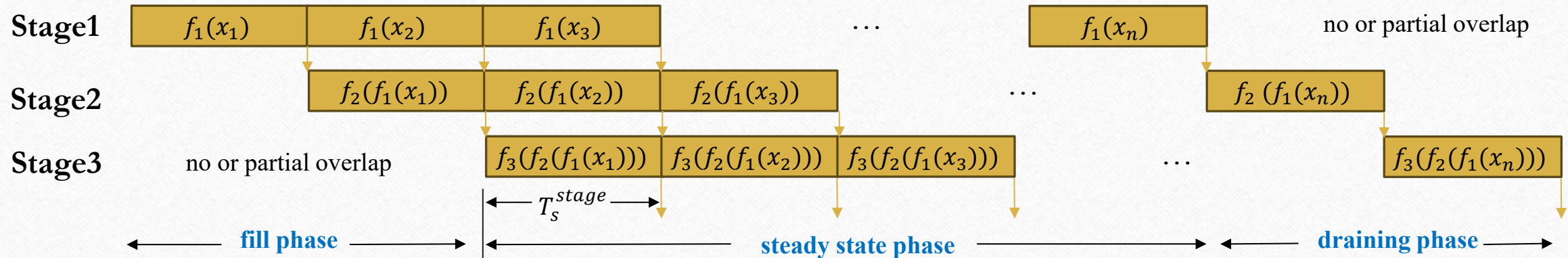


Why is it efficient?

2/2

- The throughput of the **sequential** computation of F is: $Throughput = \frac{n}{n*75}$, one task every 75 time units

Timing diagram:



- The throughput of the 3-stage pipeline is: $Throughput = \frac{n}{(n+2)*25}$, about one task every 25 time units if n is large enough (i.e., $n \gg 3$)
- The steady state throughput is $1/25$, i.e., $\frac{1}{T_s}$

What if $T_s^{stage_1} \neq T_s^{stage_2} \neq T_s^{stage_3}$?
With **unbalanced stages**, we need to do some adjustments

In this example the service time of the entire pipe is $T_s^{pipe} = T_s^{stage}$

What about the completion time?

- Let's consider again the previous example. Suppose $n = 300$.

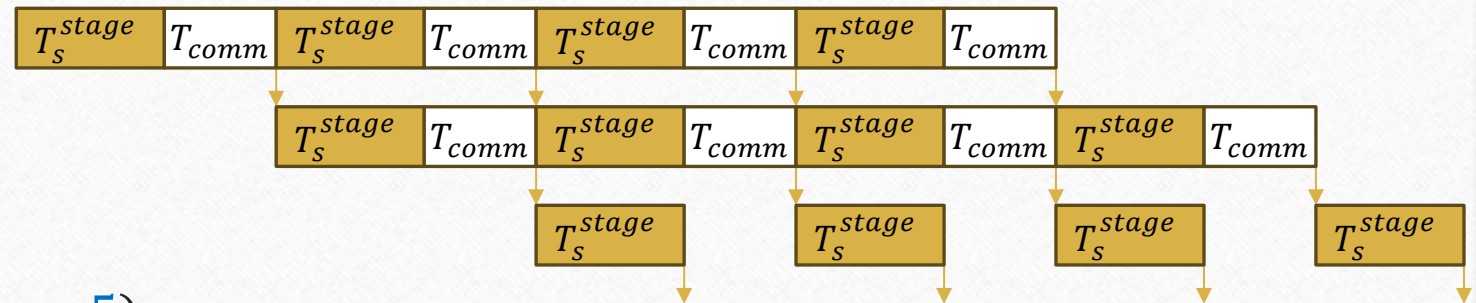
- $T_{comm} = 0$

- $T_c^{seq} = n \cdot T_s^{seq} = 22500$
- $T_c^{pipe,3}(n) = (2 + n) \cdot T_s^{stage} = 7550$
- The speedup is $S(3) = \frac{T_c^{seq}}{T_c^{pipe,3}} \cong 2.98$

- $T_{comm} \neq 0, T_{comm} < T_s$ (e.g., $T_{comm} = 5$)

- $T_c^{seq} = n \cdot T_s^{seq} = 22500$
- $T_c^{pipe,3}(n) = 2 \cdot (T_s^{stage} + T_{comm}) + (n - 1) \cdot (T_s^{stage} + T_{comm}) + T_s^{stage} = 9055$
- The speedup is $S(3) = \frac{T_c^{seq}}{T_c^{pipe,3}} \cong 2.48$

Timing diagram ($T_{comm} \neq 0$) :



Assumption: T_{comm} is a fixed hand-off cost for each stage

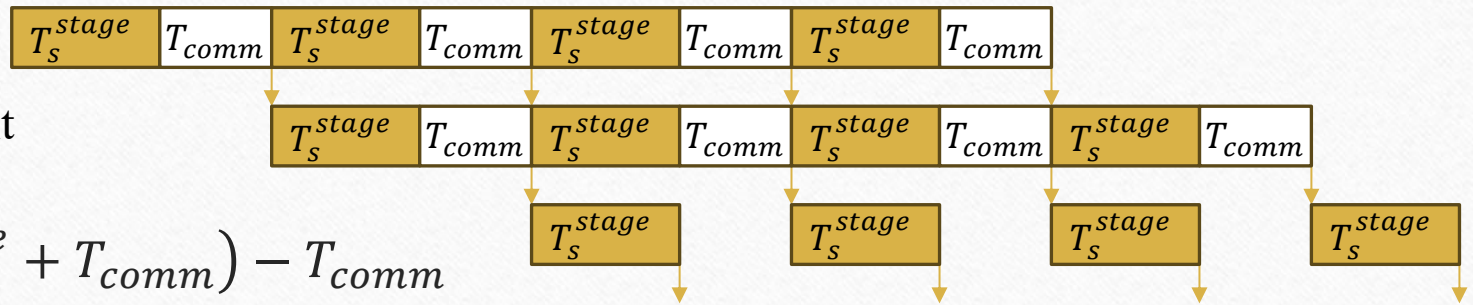
NOTE: Latency is worse than sequential version!

From $L_{seq} = T_s^{seq}$ to $L_{pipe,3} = T_s^{seq} + 2 \cdot T_{comm}$

Completion time with balanced stages

k-stage pipeline

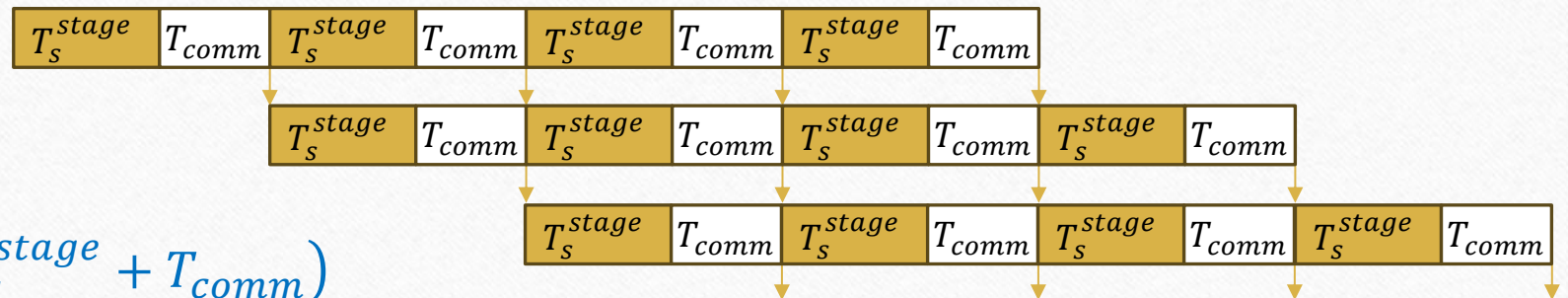
Timing diagram :



The sink stage does not send out the result

- $T_c^{pipe,k}(n) = (n + k - 1) \cdot (T_s^{stage} + T_{comm}) - T_{comm}$

Timing diagram :



The sink stage sends out the result

- $T_c^{pipe,k}(n) = (n + k - 1) \cdot (T_s^{stage} + T_{comm})$

NOTE: In the timing diagram we suppose there is no computation-to-communication overlap

What if the stages are unbalanced?

- Suppose $T_s^{stage_1} \neq T_s^{stage_2} \neq T_s^{stage_3}$. For example $T_s^{stage_1}=15$; $T_s^{stage_2} = 25$; $T_s^{stage_3} = 12$



Timing diagram:

- $T_{comm} = 5$, $n=300$

- $T_c^{seq} = n \cdot T_s^{seq} = 15600$

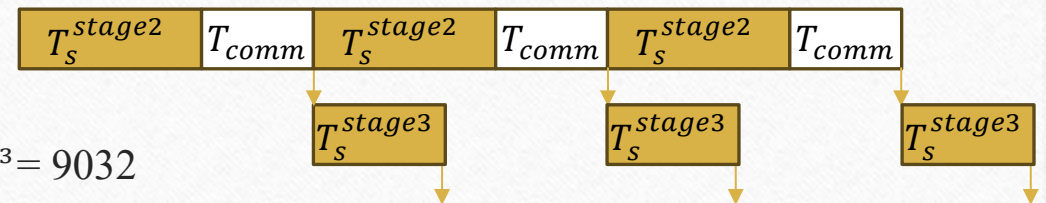
- $T_c^{pipe,3} = (T_s^{stage_1} + T_{comm}) + n \cdot (T_s^{stage_2} + T_{comm}) + T_s^{stage_3} = 9032$

- Time similar as if all stages have the service time of the slowest stage (i.e., $stage_2$)

$$T_c^{pipe,3} = (300 + 3 - 1) \cdot (25 + 5) - 5 = 9055$$

- If n is large, the service time of the pipeline is about that of the slowest stage (i.e., $stage_2$ in our example)

- In steady state the slowest stage dictates the pipeline's service time
- It produces one output every $(T_s^{stage_2} + T_{comm})$ during the steady phase
- The slowest stage is a **bottleneck stage** for the entire pipeline



$$T_c^{pipe,k}(n) \cong L_{pipe,k} + (n - 1) \cdot T_s^{pipe,k}$$

Pipeline cost model

- Given a pipeline of k stages where $T_s^{seq}(F) = \sum_{i=1}^k T_s^{stage_i}(f_i)$
- Service time:** $T_s^{pipe,k} = \max_{i=1..k} \{T_{s,comm}^{stage_i}(f_i)\}$ where $T_{s,comm}^{stage_i}(f_i) = \begin{cases} T_{f_i} + T_{comm}, & \text{no comm. overlap} \\ \max\{T_{f_i}, T_{comm}\}, & \text{with comm. overlap} \end{cases}$
- Task latency:** $L_{pipe,k} = T_s^{seq}(F) + (k - 1) \cdot T_{comm}$ (without communication-to-computation overlap)
- Completion time for n tasks:** $T_c^{pipe,k}(n) \cong L_{pipe} + (n - 1) \cdot T_s^{pipe,k}$, and if n is large: $T_c^{pipe,k}(n) \cong n \cdot T_s^{pipe,k}$
- Bottleneck condition:** $stage_i$ is a **bottleneck** if its service time is higher than the task's inter-arrival time (T_a)
$$T_{s,comm}^{stage_i}(f_i) > T_a^{stage_i}$$

If arrivals are not throttled and the input buffer is unbounded, a bottleneck stage causes its input queue to grow indefinitely. Possible fixes include reducing its cost, **parallelizing the stage**, changing the pipeline decomposition, or **applying backpressure**

Finding the minimal number of stages

- Let T_a be the inter-arrival time of stream items to a stage computing the function F . We want to increase the throughput of the stage by using a pipeline of k stages each computing f_i such that

$$T_s^{seq}(F) = \sum_{i=1}^k T_s^{stage_i}(f_i)$$

- What is the minimal value of k ?
- To avoid the pipeline being a bottleneck, in the ideal case of balanced stages, it must hold

$$\frac{T_s^{seq}(F)}{k} + T_{comm} \leq T_a$$

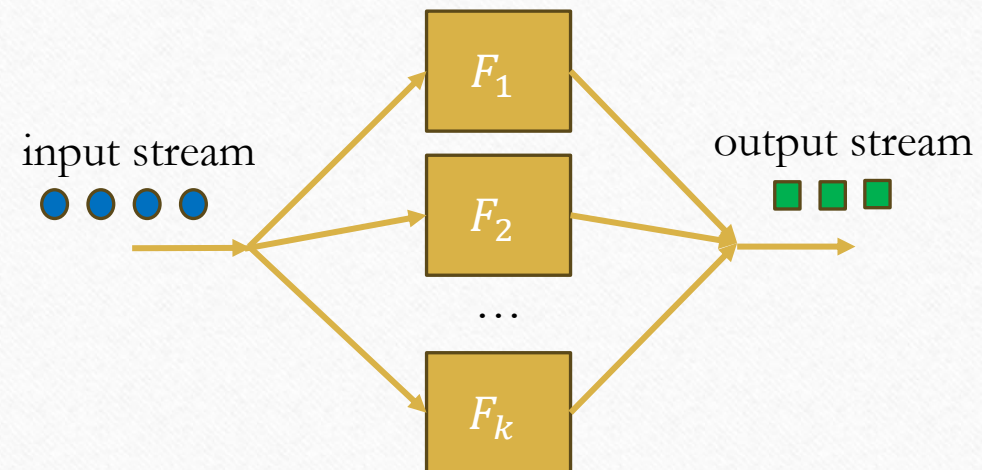
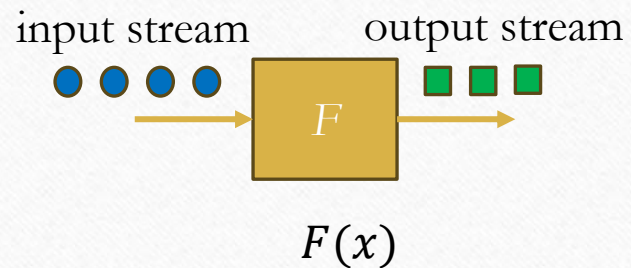
Additionally, we want to minimize the task latency $L_{pipe,k} = T_s^{seq}(F) + (k - 1) \cdot T_{comm}$

- The minimal k satisfying both equations is $k_{min} = \left\lceil \frac{T_s^{seq}(F)}{T_a - T_{comm}} \right\rceil$

Farm

1/2

- A **task farm**, often referred to simply as a **farm**, is a computing pattern that enhances computational efficiency by replicating the same (**stateless**) function F k times. Each function replica is executed by a stage called **Worker**
- Objective: **improving the throughput** of a single stage



Farm

2/2

- A bit more formally: a farm applies the same stateless function F to all items of an input stream

$$\text{farm}(F)(\|x_1, x_2, \dots\|) = \|F(x_1), F(x_2), \dots\|$$

The input is a stream of elements of type X , i.e., $\|x_1, x_2, \dots\|$, $x_i \in X$, and

$$F: X \rightarrow Y$$

thus, the type of the farm is:

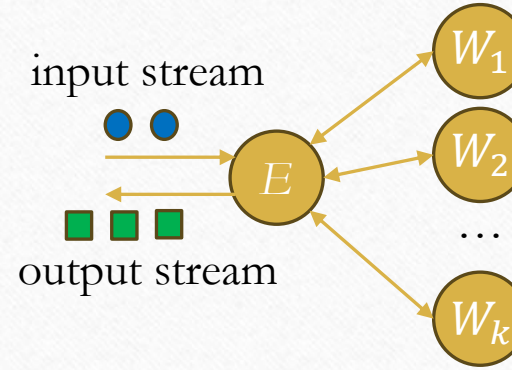
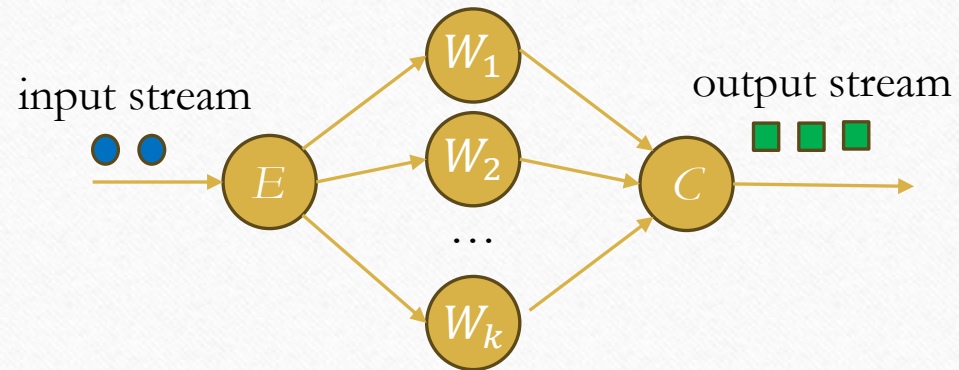
$$\text{farm}(F): \|X\| \rightarrow \|Y\|$$

- However, in general, **the input-output ordering may not need to be preserved**, thus:

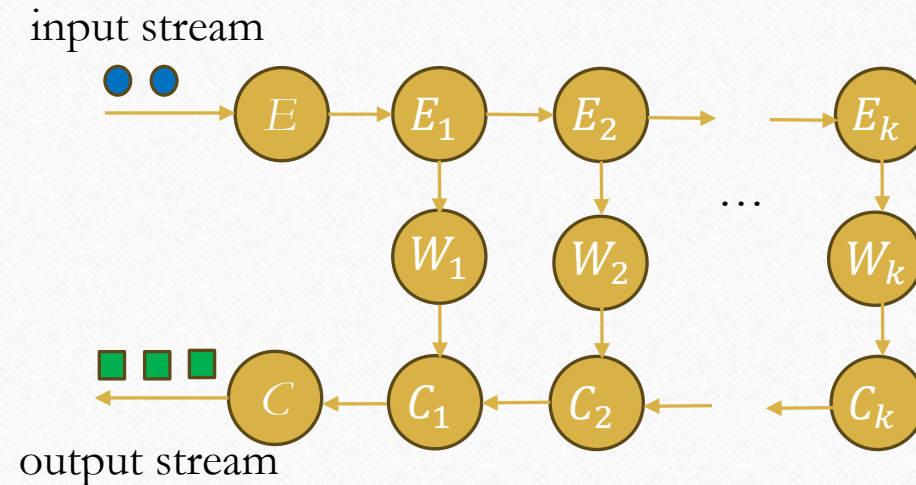
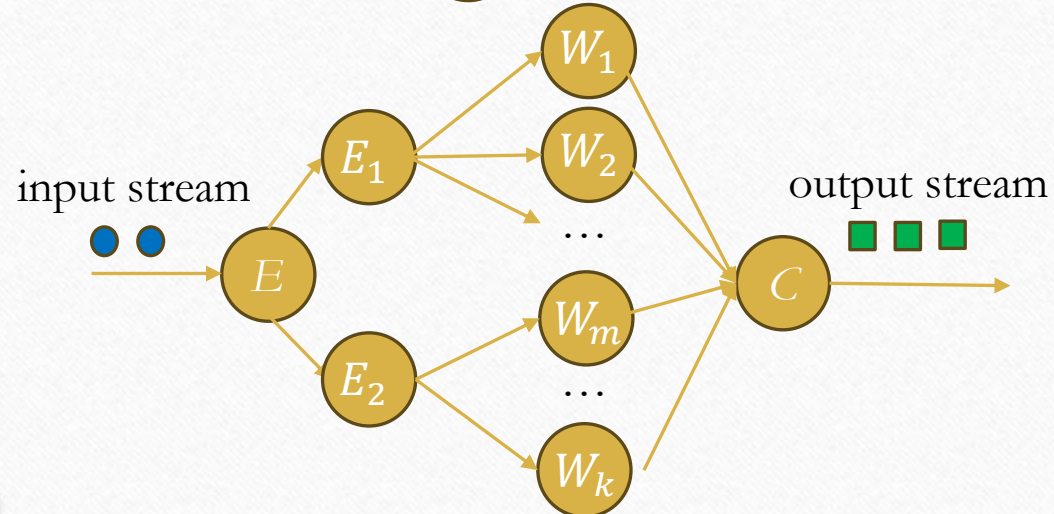
$$\text{farm}(F)(\|x_1, x_2, \dots\|) = \|\dots, F(x_{m+1}), F(x_m), \dots\|$$

NOTE: the textbook farm pattern does not preserve input-output ordering.

(Some) Farm skeletons



E stands for **E**mitter
 C stands for **C**ollector
 W stands for **W**orker



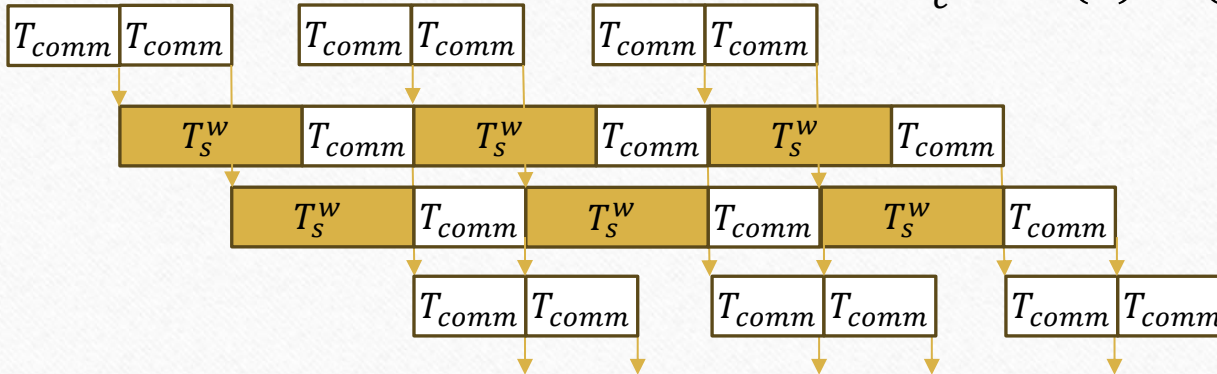
Farm completion time

- Consider an **implementation skeleton** with one Emitter and Collector. They just schedule and gather tasks $T_s^e = T_s^c$
- The farm has k Workers and $T_s^e < T_a$, $T_s^e < T_s^w$ and $T_s^c < T_a$, $T_s^c < T_s^w$
- The stream has n tasks

Timing diagram ($T_{comm} \neq 0$):

$$T_c^{farm,k}(n) = (k + 1) \cdot T_{comm} + \left\lceil \frac{n}{k} \right\rceil \cdot (T_s^w + T_{comm})$$

Emitter



T_s^e Emitter service time
 T_s^c Collector service time
 T_s^w Worker service time

In general: $T_c^{farm,k}(n) = L_{farm} + (n - 1) \cdot T_s^{farm,k}$ where $T_s^{farm,k} = \max(T_s^e, T_s^c, \frac{T_{s,comm}^w}{k})$

Finding the number of Workers

- Let T_a be the inter-arrival time of tasks to a farm computing the function F with k Workers
- What is the minimal value of k ?
- Assuming Emitter and Collector are not bottlenecks $T_s^e < T_a$ and $T_s^c < T_a$, it must hold

$$\frac{T_{s,comm}^w(F)}{k} \leq T_a$$

and to avoid over-provisioning of Workers, k should not exceed

$$k \leq \frac{T_{s,comm}^w(F)}{\max\{T_s^e, T_s^c\}}$$

- The minimal k that avoids Workers stage being a bottleneck is:

$$k_{min} = \left\lceil \frac{T_{s,comm}^w(F)}{T_a} \right\rceil$$

Adding more worker than this upper bound does not improve the throughput, because Emitter and Collector service time dominate

Farm cost model

- Given a farm with k Workers, the Worker service time is

$$T_{s,comm}^w(F) = \begin{cases} T_s^{seq}(F) + T_{comm} \\ \max\{T_s^{seq}(F), T_{comm}\} \end{cases}$$

no comm. overlap
with comm. overlap

- One Emitter and Collector stages, with their service time T_s^e, T_s^c

- The farm service time is:** $T_s^{farm,k}(F) = \max\left\{\frac{T_{s,comm}^w(F)}{k}, T_s^e, T_s^c\right\}$

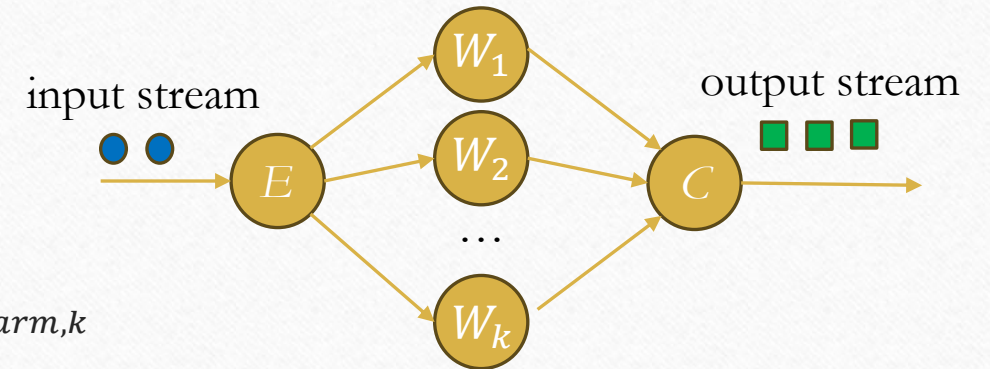
- Task Latency:** $L_{farm} = T_s^e + T_{s,comm}^w(F) + T_s^c$

- Completion time for n tasks:** $T_c^{farm,k}(n) = L_{farm} + (n - 1) \cdot T_s^{farm,k}$

- Bottleneck condition:** the farm is a **bottleneck** if its service time is higher than the task's inter-arrival time (T_a)

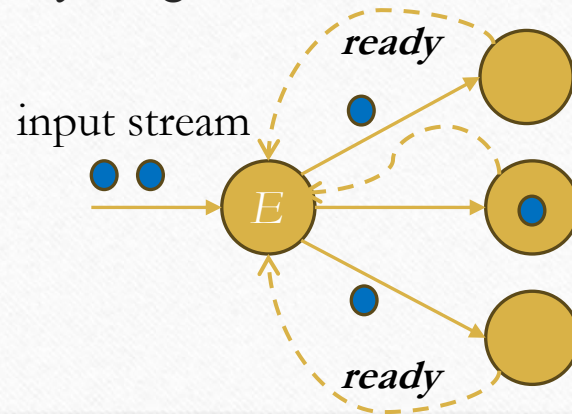
$$T_s^{farm,k}(F) > T_a$$

If $T_s^e > T_a$ or $T_s^c > T_a$, increasing k cannot remove the bottleneck



Task scheduling: active Emitter policies

- The Emitter must assign input tasks to Workers to ensure an even workload distribution
- If $T_F(x_i) \cong T_F(x_j)$ for all i, j , then the **round-robin (RR)** task scheduling strategy is an effective policy
 - Input x_i goes to Worker W_j such that $j = i \bmod k$ where k is the number of Workers
 - Tasks are distributed evenly among Workers in a cyclic manner
- If task costs are irregular or unpredictable, the **on-demand (OD)** task scheduling can be used
 - each input task is dynamically assigned to Workers as they become available

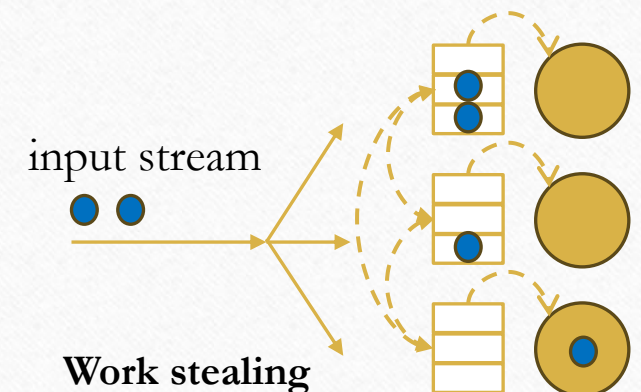
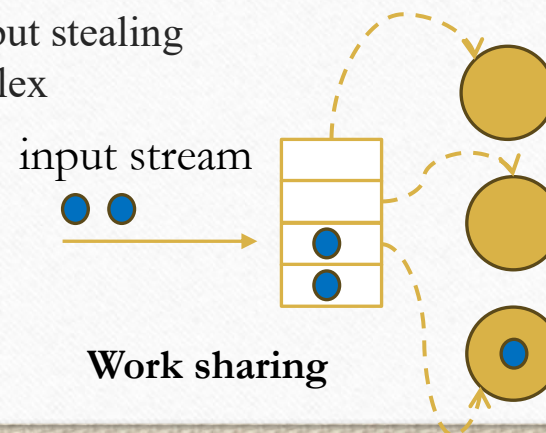


NOTE:

- *Ready* is a control message sent by a Worker to the Emitter to request a new task
- In pure OD scheduling, the Worker sends the *ready* message when it becomes available
- If Worker can buffer tasks, the Worker may send the *ready* message earlier as soon as it receives a task to compute, so overlap computation and communication

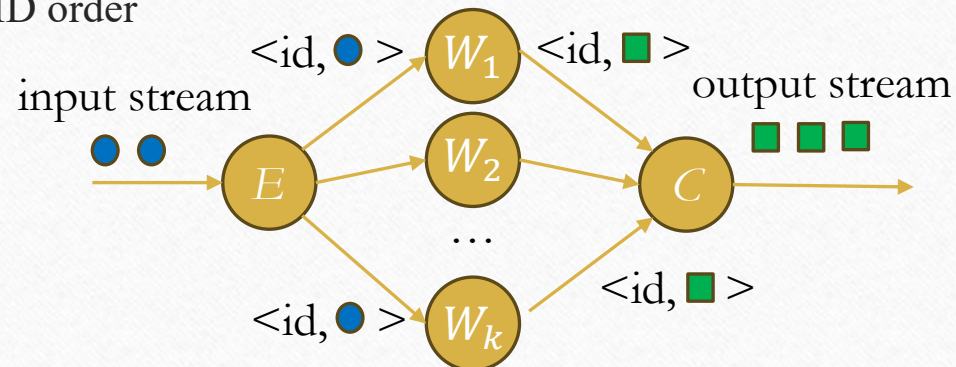
Task scheduling: passive queue-based policies

- The Emitter can be replaced by **passive scheduling structures** (one or more concurrent task queues)
 - The concurrent queues can be implemented using locks or in a lock-free way
- **Work-Sharing**: the Workers pull tasks from a single shared concurrent task queue
 - It is simple and naturally load-balanced, but the shared queue may become a contention point
- **Work-Stealing**: each Worker owns its local task queue; tasks may initially be assigned to Workers using a RR or random policy; idle Workers (i.e., those with empty local queue) try to steal (a batch of) tasks from other Workers
 - It reduces contention and improves locality, but stealing introduces overhead and requires more complex termination detection



Preserving input ordering

- What if input-output ordering must be preserved when using a farm pattern?
 - Example: video streaming where tasks are video frames; they can be processed independently but the order must be preserved in the output video
- Two possible strategies:
 - **Deterministic scheduling and collection of tasks**: the exact same order is used in the scheduling and gathering policies.
 - What kind of problem may we face?
 - The Emitter stage adds a tag (an ID) to each scheduled task and the Collector stage removes the tag and produces the tasks in output based on the ID order



Preserving ordering may introduce **head-of-line blocking**: a slow task with a small ID delays the emission of later completed tasks. The Collector must **buffer** out-of-order results

What if F is stateful?

1/2

- Traditionally, farm patterns assume **stateless** Workers: each task is processed independently, and no internal data persists across tasks. However, some applications require **stateful** computations
- **Read-only state**
 - **Distributed Memory**: each Worker holds a local replica of the read-only data. There is no need for synchronization beyond the initial data distribution
 - **Shared Memory**: Workers can access a shared read-only copy of the state
- For **Read-Write state**, we distinguish between
 - **Monolithic (non-partitionable) state**: updates affect a single global logical state. Workers must synchronize access to the shared state, or keep replicas consistent
 - **Partitionable state**: divide the global state into **disjoint** segments so each worker handles a portion of the state

What if F is stateful?

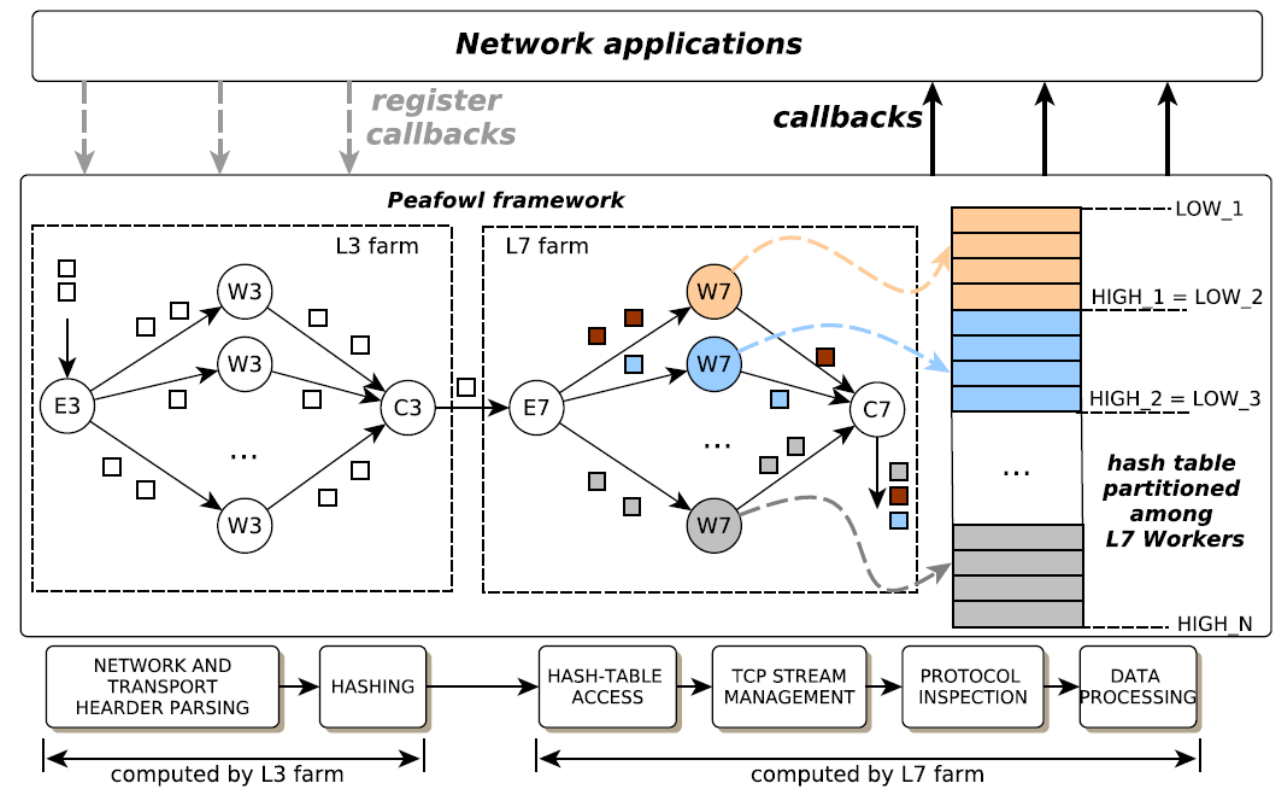
2/2

- Non-partitionable state
 - The performance depends on the ratio of writes operations to reads operations. The synchronization or replication cost grows with the frequency of writes
 - The multiple-reader single-writer locking scheme can be used on shared-memory systems
 - Multicast-based approach in distributed-memory systems: tasks that update the state are broadcasted to all Workers to update their local state (or, equivalently, the updated state is broadcasted to all Workers)
 - Updates must be applied in the same logical order on all replicas
- Partitionable state
 - Apply a hash function to each task to identify the state partition, then send the task to the corresponding Worker
 - Workers are not anonymous. The Emitter must route each task to the Worker that owns the corresponding state partition

Stateful Farm: partitionable state example

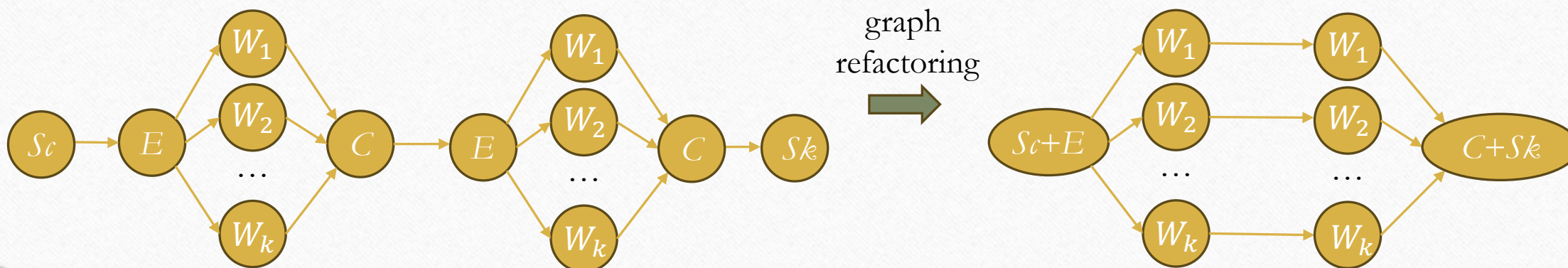
- Deep-packet inspection with per-flow state

- The Emitter computes a hash on the flow key, for example the 5-tuple:
 - <source IP, destination IP, source port, destination port, protocol>
- The hash identifies the Worker responsible for that flow:
 - $\text{worker_id} = \text{hash}(\text{flow_key}) \bmod k$
- This preserves per-flow locality and avoids synchronization on the connection state
- Issue: load balancing is only probabilistic, but works well in practice
 - A good hash function distributes many independent flows reasonably well, but it balances the number of flows, not necessarily the amount of traffic
 - A few high-volume flows may concentrate on the same Worker, creating load imbalance.



Nesting Pipelines and Farms

- Nesting allows programmers to break down a complex application into multiple layers of concurrently executed stages. For example, the application is structured as a pipeline sequence of operations while, within each stage, a farm is used to process several independent tasks in parallel and increase throughput
- However, extra layers produced by the nesting, may introduce additional communication and synchronization overheads, thus **increasing the end-to-end latency**



What if we don't have a stream of tasks?

- Streams can be either primitive (“**exo-stream**”), i.e. , coming from an authentic source, for example, a sensor, a network interface, a DB, etc., or can be generated directly by the application (“**endo-streams**”), usually by producing the stream from loop iterations
- A stream can be obtained from a monolithic collection by splitting the collection in sub-collections and then re-collecting the results in a single collection
 - **SplitCollection** → **Compute** → **Collect** pattern
 - Producing a stream of tasks from loop iterations (e.g., **taskloop** in OpenMP)
 - Pipeline and Farm can be used also with endo-streams
 - Producing a stream artificially, is not always the best option. Extra overhead due to communication can be introduced

Example

1/4

- Let's consider the following simple sequential program:

```
while(EOFcondition(inFile)) {  
  X=readFromFile(inFile);  
  Y=F(X);  
  Z=G(Y);  
  writeToFile(outFile, Z);  
}
```

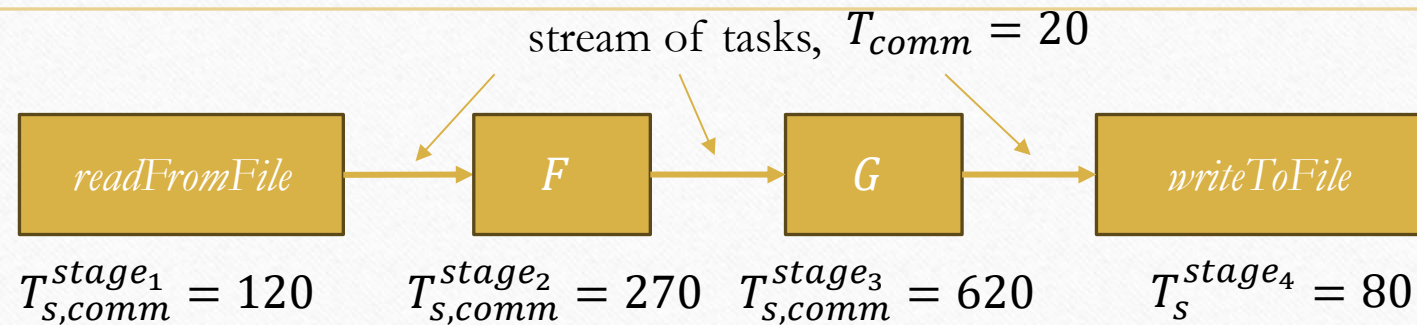
- readFromFile* takes 100 time units; *writeToFile* takes 80 time units
 - F* takes 250 time units, and *G* 600 time units;
 - F* and *G* can be replicated whereas I/O functions are difficult to parallelize
 - $T_s^{seq} = 1030$ time units per task
- We want to reduce the completion time ($T_c^{seq} = n \times 1030$) by parallelizing the code
 - We can assume that the number of tasks (n) to be completed is large enough
 - When moving to a parallel version, the cost of communication per task is 20 time units
- What parallelization options do we have?

Example

2/4

- First option:

- 4-stage pipeline



$$T_c^{pipe,4}(n) = L_{pipe,4} + (n - 1) \cdot T_s^{pipe,4} \cong n \cdot \max_{i=1..4} \{T_s^{stage_i}\} = n \cdot 620$$

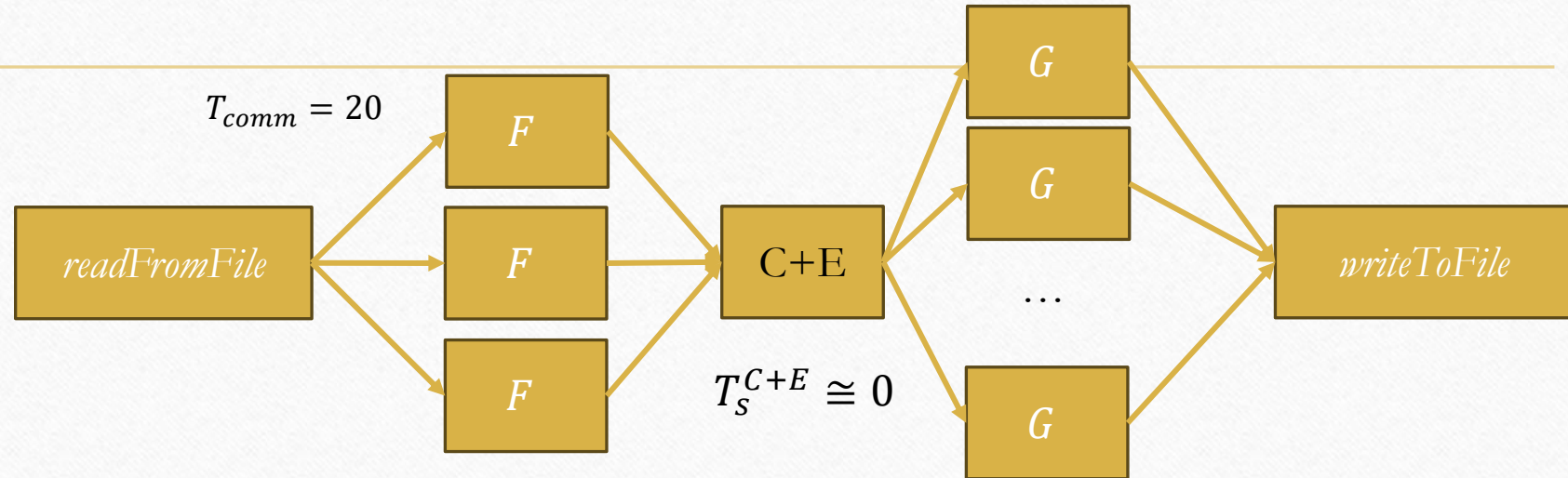
- Both F (270) and G (620) are bottlenecks

- We can use a farm pattern for both to reduce their service time since F and G can be replicated

Example

3/4

- Second option:
 - Pipe of farms



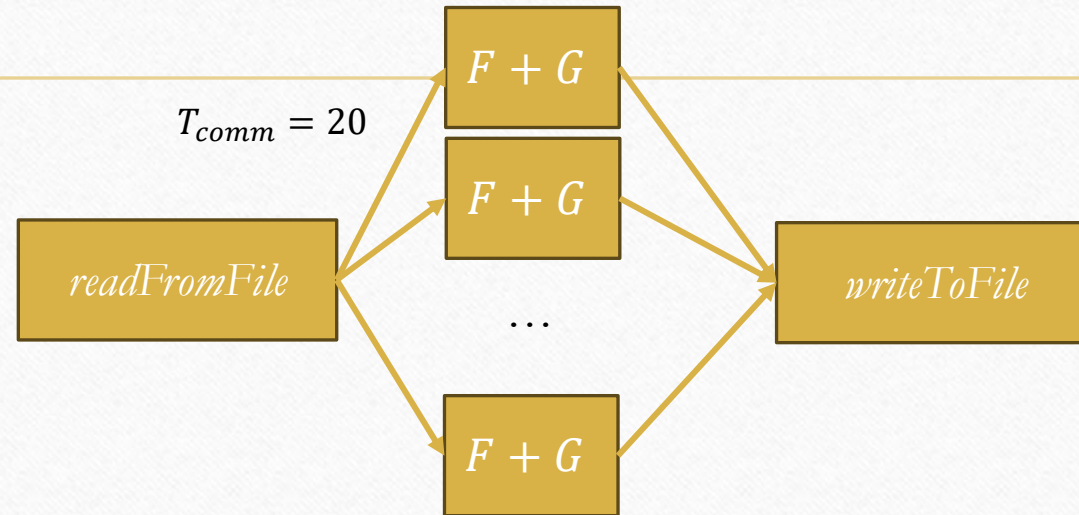
- $T_a^{stage_2} = 120$ therefore $k_{min}^{stage_2} = \left\lceil \frac{270}{120} \right\rceil = 3$ and the new service time is $T_s^{stage_2} = \frac{270}{3} = 90$
- $T_a^{stage_3} = \max\{T_s^{stage_1}, T_s^{stage_2}\} = 120$ therefore $k_{min}^{stage_3} = \left\lceil \frac{620}{120} \right\rceil = 6$ and $T_s^{stage_3} = \frac{620}{6} \cong 104$
- The bottlenecks have been removed, now the new bottleneck is the first stage that we cannot parallelize
- $T_c^{pipe}(n, k) \cong n \cdot 120$
- The speedup is $S(12) = \frac{T_c^{seq}(n)}{T_c^{pipe,12}(n)} \cong 8.6$ (~71% efficiency)

NOTE: Latency increases from 1030 (sequential) to 1110, i.e., it increases by $4 \cdot T_{comm}$

Example

4/4

- Third option:
 - Single farm («normal form»)



- $T_a^{stage_{2+3}} = 120$ therefore $k_{opt}^{stage_{2+3}} = \left\lceil \frac{870}{120} \right\rceil = 8$ and the new service time is $T_s^{stage_{2+3}} = \frac{870}{8} \cong 109$
- The completion time is about the same $T_c^{pipe,k}(n) \cong n \times 120$ but latency is lower and we use less resources
 - Efficiency increases from ~71% to 86%

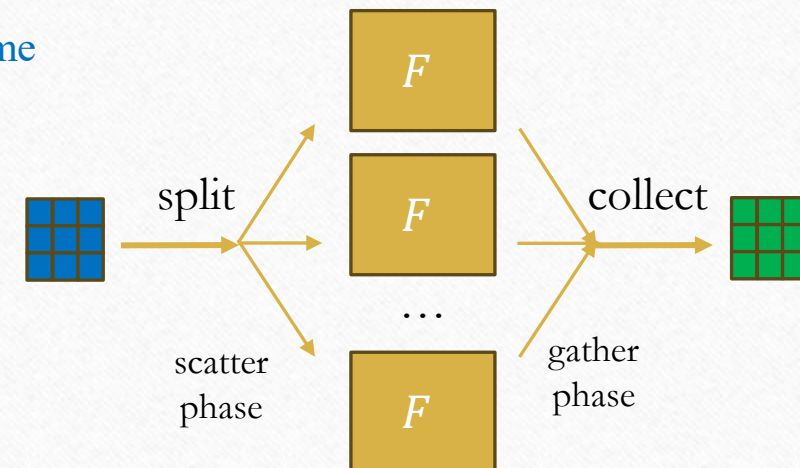
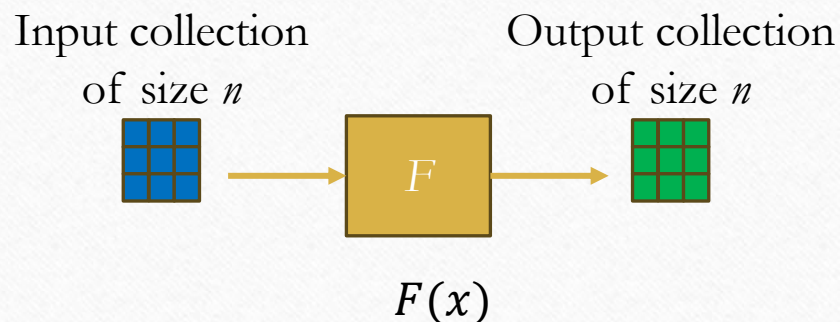
Data parallelism (recap)

- Data parallelism considers an input **collection** of data and produces an output collection of data
 - In general, input and output collection sizes may differ
- **The same operation (or kernel function) is applied to data elements (or blocks of data elements) in parallel on multiple Workers**
- The primary data parallel patterns are (input : output cardinality):
 - **Map** (N : N) - example: convert a list of temperatures from Celsius to Fahrenheit
 - **Reduce** (N : 1 or N : M with $M \ll N$) - example: sum N numbers, or counts per class (M classes)
 - **Stencil** (N : N) - example: 2D convolution
 - **Scan** (N : N) - example: cumulative sums of N numbers
- Objective: **reduce the completion time**

Map

1/2

- A **map** is a data-parallel pattern in which a single function F can be applied independently to each element of an input **collection**, producing an output collection **of the same cardinality**
- The parallelism is exploited by using a pool of Workers, each working independently on a partition of the input collection and producing a partition of the output collection
- The map pattern aims to decrease the completion time/latency for one input collection
 - If there is a stream of collections, a parallel map lowers the service time and increases throughput



Map

2/2

- A bit more formally: given a collection of data elements $D = \{d_1, \dots, d_n\}$ and a function F , the **map operation** is defined as:

$$\text{map}(F)(D) = \{F(d_1), \dots, F(d_n)\}$$

The elements of the collection have type X , i.e., $d_i \in X$, and

$$F: X \rightarrow Y$$

therefore, the type of the map is:

$$\text{map}(F): \{X\} \rightarrow \{Y\}$$

Map implementation

- A map can be implemented using a **farm skeleton** (e.g., Emitter - Workers - Collector) where each Worker applies F to a partition of the input collection
- The main issues are **workload balancing** and the cost of distributing/collecting data
- If the workload associated with the input collection is balanced, the Emitter may **scatter** contiguous blocks of the collection to Workers
 - In this case the map pattern is also called **embarrassingly parallel computation**
- If the workload is unbalanced or unpredictable, a dynamic approach can be used:
 - work-sharing or work-stealing (**passive approach**)
 - on-demand distribution coordinated by the Emitter (**active approach**)

Map cost model

- Given a collection of n elements; k Workers, one Emitter and Collector stages

- Completion time** (point-to-point scatter and gather, Farm skeleton):

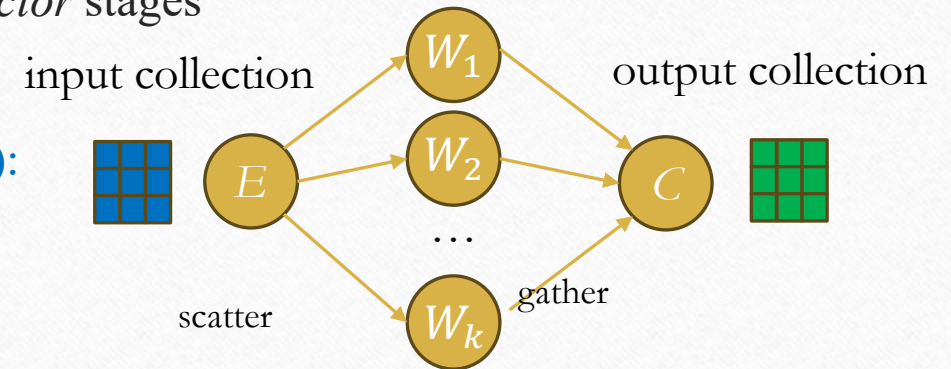
$$T_c^{map}(n, k) = T_{scatter} + T_{worker} + T_{gather} \quad \text{where}$$

$$T_{scatter} = T_{split(n,k)} + k \cdot T_{comm}\left(\frac{n}{k}\right)$$

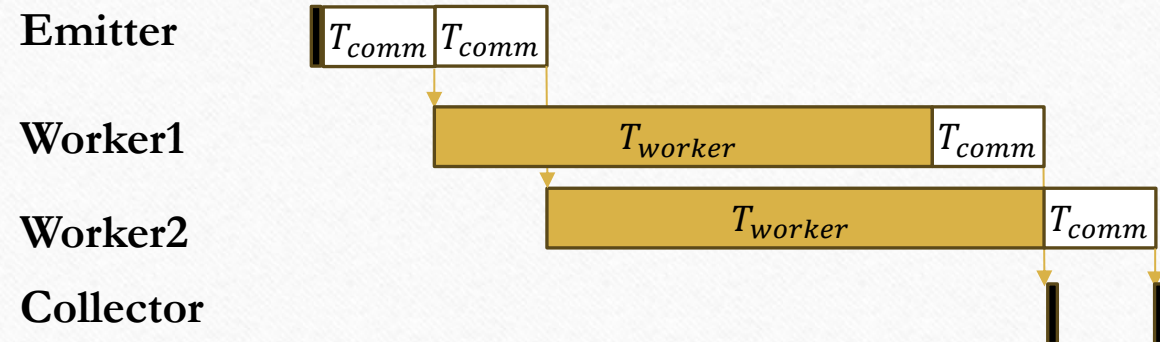
$$T_{worker} = \frac{n}{k} \cdot T_F + T_{comm}\left(\frac{n}{k}\right)$$

$$T_{gather} = T_{gather}\left(\frac{n}{k}\right)$$

NOTE: The **pipeline parallelism** between Emitter and Workers and between Workers and Collector allows for partial overlap of the cost of communications



Timing diagram :

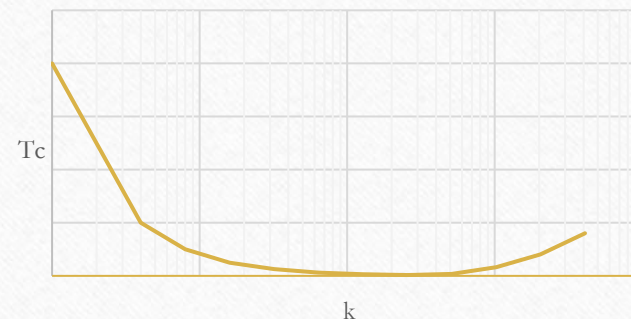


Finding the number of Workers

- $T_c^{map}(n, k)$ depends on k , so what is the optimal value of k that minimizes completion time?
- Considering the linear communication cost model, i.e., $T_{comm}(n) = t_0 + n \cdot s$ and assuming that $T_{split}(n, k) \cong 0$ and $T_{gather}(\frac{n}{k}) \cong 0$, we have:

$$T_c^{map}(n, k) = k \cdot T_{comm}\left(\frac{n}{k}\right) + \frac{n}{k} \cdot T_F + T_{comm}\left(\frac{n}{k}\right) =$$
$$(k + 1) \cdot T_{comm}\left(\frac{n}{k}\right) + \frac{n}{k} \cdot T_F = (k + 1) \cdot \left(t_0 + \frac{n}{k} \cdot s\right) + \frac{n}{k} \cdot T_F$$

- $\frac{d}{dk} T_c^{map}(n, k) = 0$ for $k_{opt} = \left\lceil \sqrt{n \cdot \frac{(T_F + s)}{t_0}} \right\rceil$
 - k_{opt} is (usually) a very large number because n is large and t_0 is small



Example:

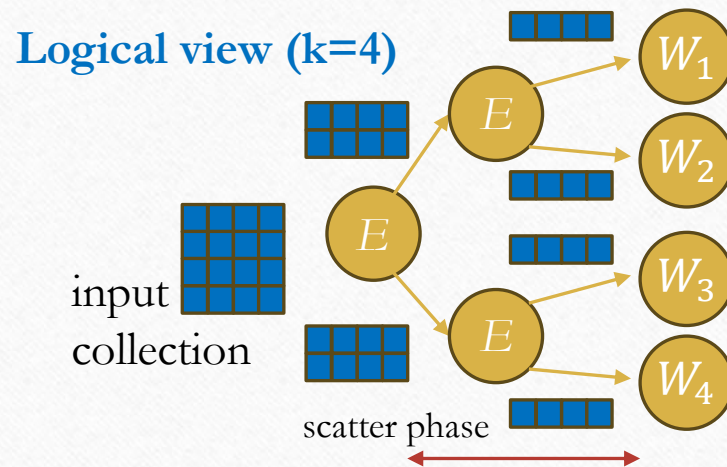
$t_0 = 2 \mu s$, $BW = 200 MB/s$, $n = 100M$ elements
 $T_F = 1 \mu s$, $s = \frac{8 \text{ byte}}{200 MB/s} = 0.04 \mu s$

$$k_{opt} = 7211 \quad T_c^{seq} \cong 100s$$

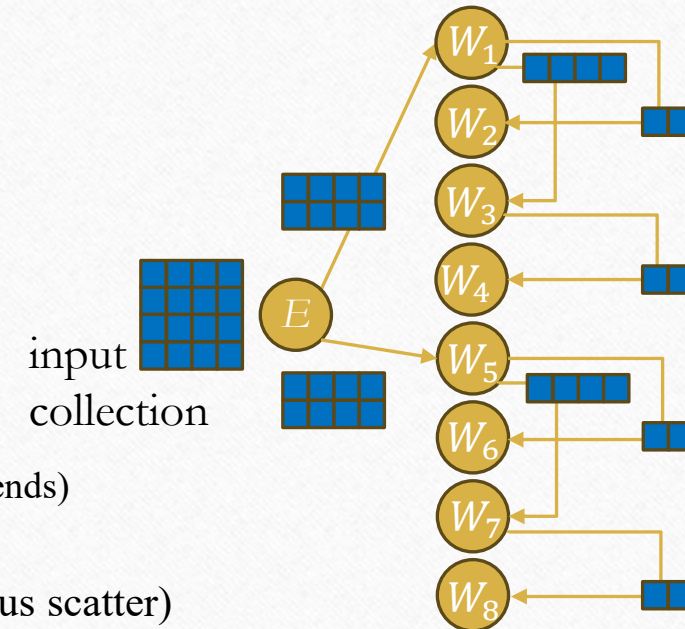
$$T_c^{map}(1024) \cong 4.10s \quad T_c^{map}(k_{opt}) \cong 4.02s$$

Binary tree-like scatter

- To reduce both the service time of the Emitter and the overall latency of the map to complete the scatter distribution, a binary tree-like scatter distribution can be employed



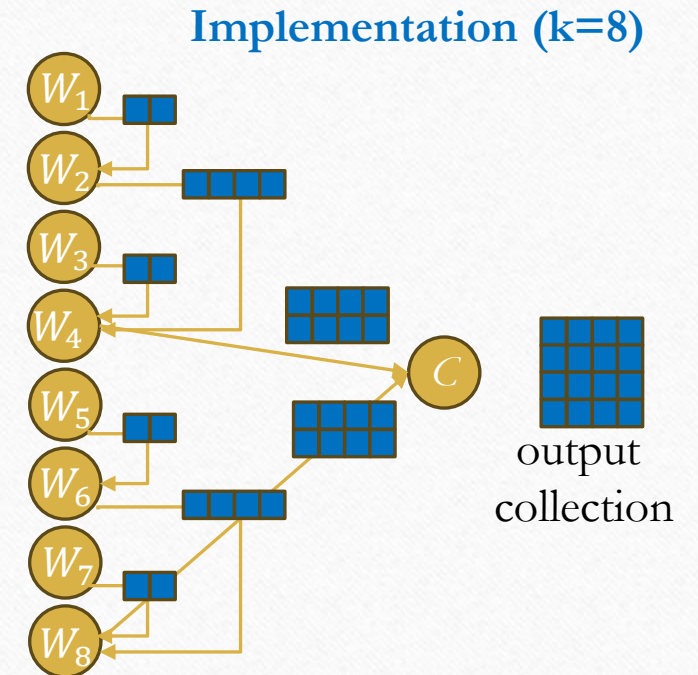
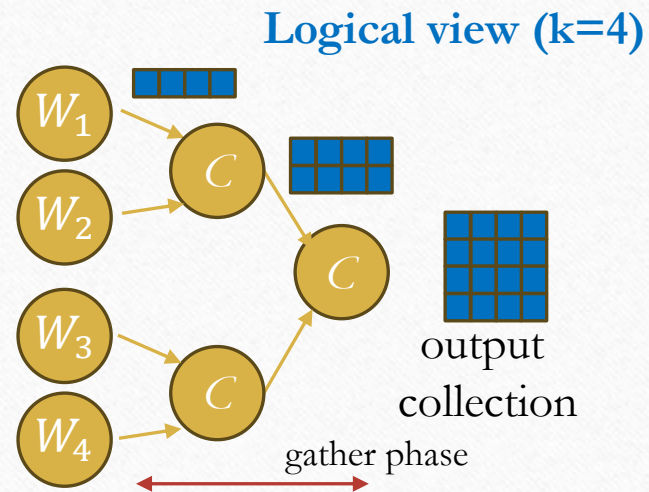
Implementation (k=8)



- Emitter service time:** $T_s^E = 2 \cdot T_{comm}(\frac{n}{2})$ (no overlap between the 2 sends)
- Scatter latency:** $T_{scatter} = 2 \cdot \sum_{i=1}^{\lceil \log_2 k \rceil} T_{comm}(\frac{n}{2^i})$ (level-synchronous scatter)

Binary tree-like gather

- To reduce both the Collector's service time and the end-to-end gather latency, a **binary tree-like gather collection** can be employed



- Collector service time:** $T_s^C = 2 \cdot T_{comm} \left(\frac{n}{2} \right)$ (no overlap between the 2 receives)
- Gather latency:** $T_{gather} = 2 \cdot \sum_{i=1}^{\lceil \log_2 k \rceil} T_{comm} \left(\frac{n}{2^i} \right)$ (level-synchronous gather)

Map cost-model leveraging a Parallel Filesystem

1/2

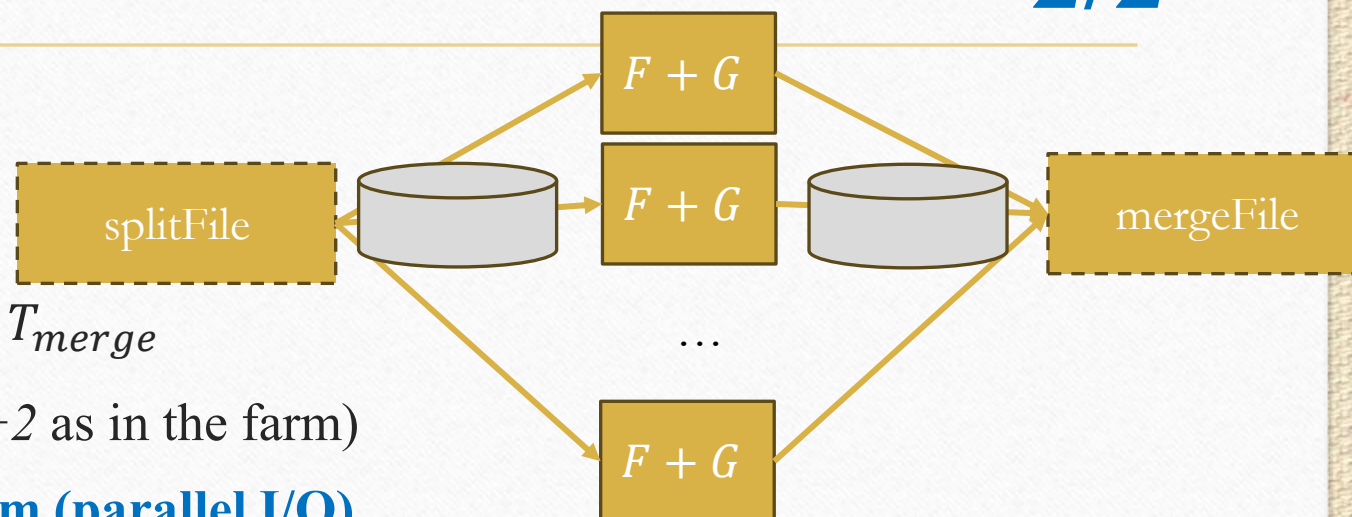
- Let's consider again the sequential program in the side picture
- Since all tasks are available in advance and they are independent, we can view them as a collection, and therefore we can apply the Map pattern
- The input file can be split into k sub-collections, and each Worker can independently read and work on one sub-collection
- Sub-collections must be merged into a single collection
 - Often, when size and offset of the output partitions are known, Worker can write directly to independent regions of a single output file without conflict, for example using pwrite or MPI-IO

```
while(!EOFcondition(inFile)) {  
    X=readFromFile(inFile);  
    Y=F(X);  
    Z=G(Y);  
    writeToFile(outFile, Z);  
}
```

```
Partitions= splitFile(inFile, k);  
foreach i in Partitions  
    while(!EOFcondition(Partitions[i])) {  
        X=readFromFile(Partitions[i]);  
        Y=F(X);  
        Z=G(Y);  
        writeToFile(outPartitions[i], Z);  
    }  
collectFile(outPartitions, k, outFile);
```


Map cost-model leveraging a Parallel Filesystem

2/2

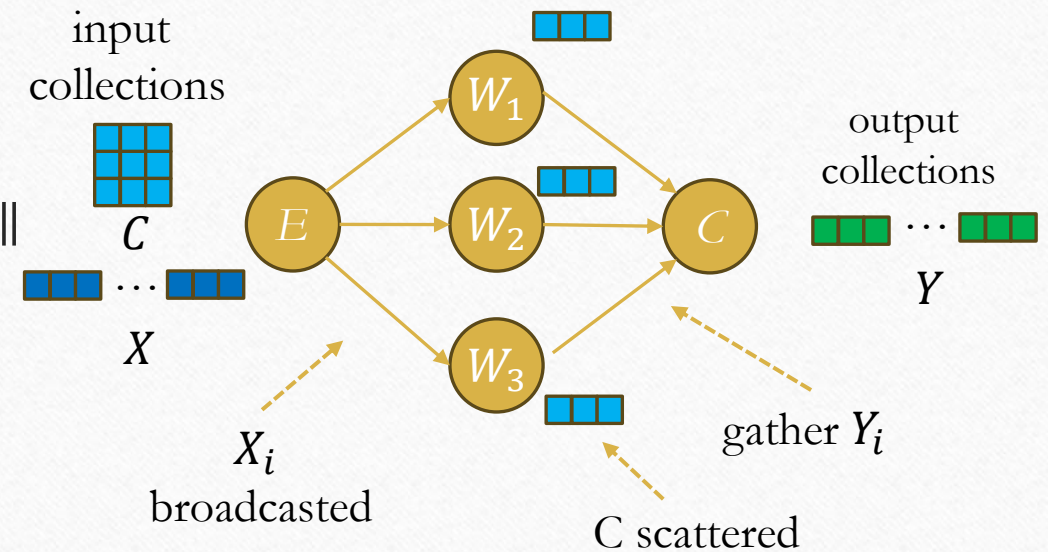


- $T_c^{map,k}(n) = T_{split} + T_{IO} \left(\frac{n}{k} \right) + \frac{n}{k} \cdot T_{F+G} + T_{merge}$
- In this case, we use only k resources (not $k+2$ as in the farm)
- **Potential advantage of a parallel filesystem (parallel I/O)**
 - If the data is already distributed, the splitting overhead is minimal
 - Splitting and merging are logical operations that do not actually move data
 - In this scenario, $T_c^{map,k}(n) \approx T_{IO} \left(\frac{n}{k} \right) + \frac{n}{k} \cdot T_{F+G}$

NOTE: The Hadoop's HDFS maintains a logical view, while its physical organization promotes efficient scatter/gather-like operations during MapReduce computations.

Map specializations

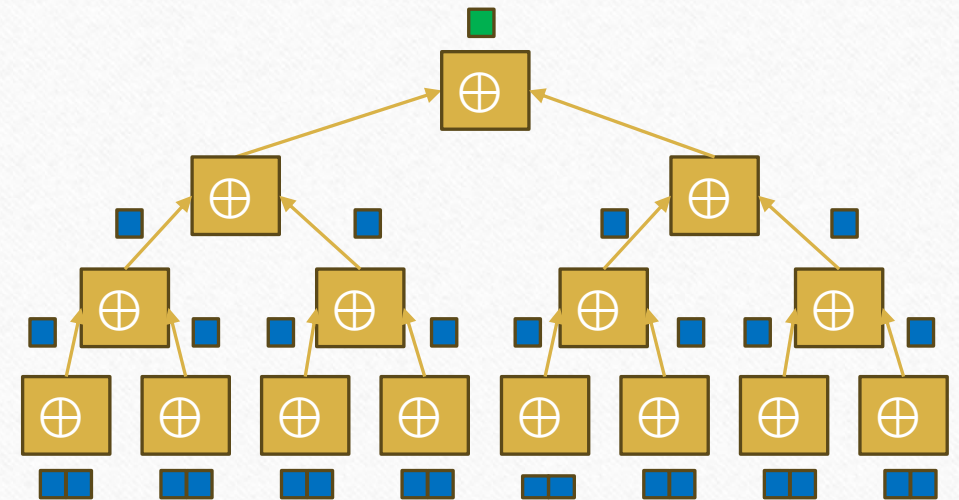
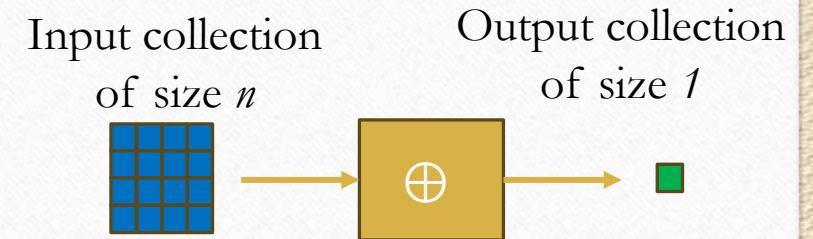
- A **map** pattern usually partitions input collections following the schema **Scatter - Compute - Gather**
- However, there are cases where the input collections need to be distributed differently (e.g., Broadcast+Compute+Gather)
- One paradigmatic case is when we have:
 - One input collection C (**partitioned** among Workers)
 - A second input collection of elements $X = \|X_1, X_2, \dots, X_k\|$
 - A Worker function F such that $Y_i = F(C, X_i)$
 - A stream of output collections $Y = \|Y_1, Y_2, \dots, Y_k\|$ where $Y_i = \{y_1, \dots, y_{\#C}\}$
- Example: Matrix-vector computation



Reduce

1/2

- A **reduce** is a data-parallel pattern that combines the elements of an input collection into a single output using an associative binary operator.
 - $x = \text{reduce}(C, \oplus)$, where $\oplus$ is the aggregate **associative** operator
 - **Commutativity** gives more freedom letting the runtime to reorder operands freely
- The parallelism may be exploited by arranging the computation as a balanced tree. The leaves combine pairs of inputs (e.g., even and odd items), then internal nodes apply $\oplus$ to the results of their left and right subtrees, all the way up to the root node



Depth of the balanced tree is $O(\log n)$

Reduce

2/2

- A bit more formally: given a collection of data elements $D = \{d_1, \dots, d_n\}$ and an associative binary operator $\oplus$, the reduce function is defined as:

$$\text{reduce}(\oplus)(D) = d_1 \oplus d_2 \oplus \dots \oplus d_n$$

The elements of the collection have type X , i.e., $d_i \in X$, and

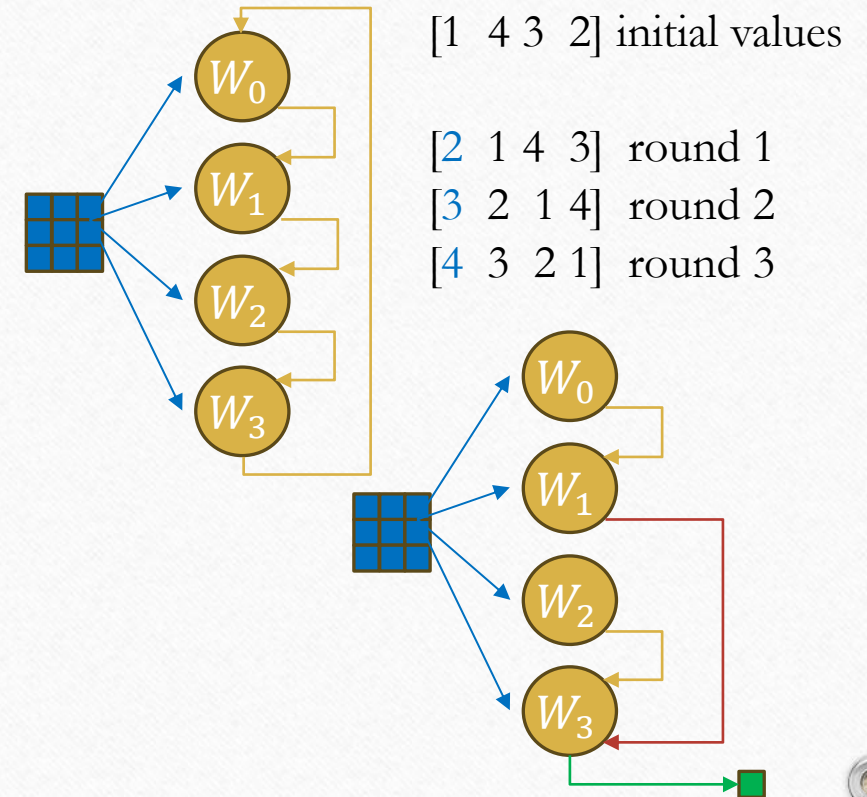
$$\oplus: X \times X \rightarrow X$$

therefore, the type of the reduce is:

$$\text{reduce}(\oplus): \{X\} \rightarrow X$$

Reduce implementations

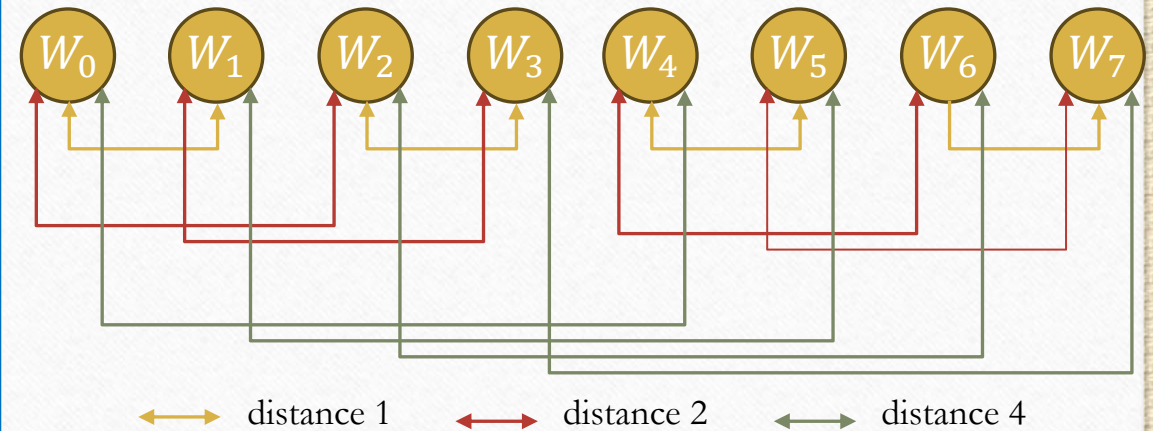
- The input collection is partitioned among all k Workers, each Worker computes the reduce on the local partition
- **Ring-based (All-)Reduce ($\oplus$ associative)**
 - Each Worker circulates its local value around the ring
 - At each round, the received value is accumulated into the partial result
 - After $k-1$ rounds, all Workers have the final reduced value locally
 - Associativity is required
 - Commutativity is needed if Workers accumulate values in cyclic order
 - Suitable for large, bandwidth-bound messages
- **Tree-based Reduce ($\oplus$ both associative and commutative)**
 - Partial results are combined along a reduction tree
 - Only the root Worker obtains the final reduced value
 - Associativity is enough if the tree preserves the input order
 - Commutativity gives more freedom to reorder operands



All-Reduce

- At the end of the All-Reduce each Worker has the reduced value locally
- Implementation of the All-Reduce with the **recursive doubling algorithm** ($\log_2 k$ communication rounds)

```
void all_reduce(id, k, localdata,  $\oplus$ ) { // assume  $\oplus$  also commutative
    distance=1;           // initial round
    pr = localdata;       // store local data into the partial result
    while( distance < k) { //  $\log_2 k$  rounds, k power of 2
        partner = id ^ distance; // find the partner (^ is the bit XOR)
        send-receive(partner, pr, r); // exchange with the partner; it sends pr to
                                      // and receives r from the partner
        pr = pr  $\oplus$  r; // computes partial result, for non-commutative operators,
                       // we must preserve a consistent rank order
        distance <<=1; // next power of 2
    }
    localdata = pr;       // final value
}
```



In some MPI implementations, this algorithm is used if k is a power of two, and for small-to-medium messages

- For non power-of-two k , implementations use a preliminary reduction step to handle extra ranks, then apply recursive doubling

Reduce cost model

- Given a collection of n elements; k Workers and the reduce operator $\oplus$

- Completion time:**

$$T_c^{reduce,k}(n) = T_{scatter} + T_{local} + T_{*-reduce} \quad \text{where}$$

$$T_{scatter} = T_{split(n,k)} + k \cdot T_{comm}\left(\frac{n}{k}\right)$$

$$T_{local} = \left(\frac{n}{k} - 1\right) \cdot T_{\oplus}$$

$$T_{tree-reduce} = \log_2 k \cdot (T_{\oplus} + T_{comm}(m)) \quad (m \text{ is the size of partial result})$$

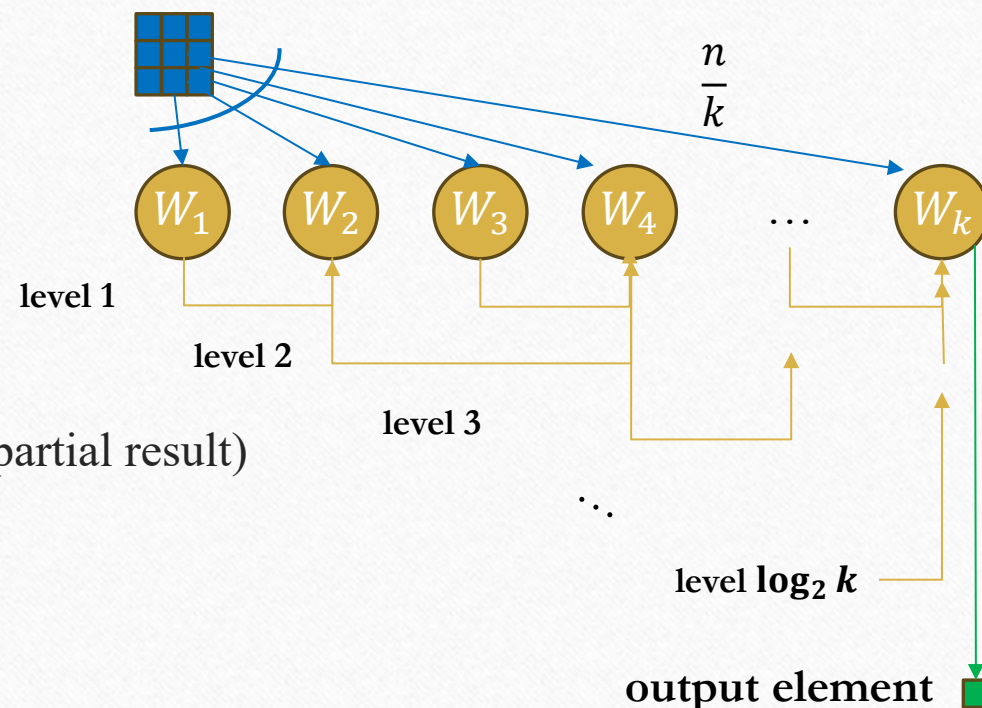
$$T_{all-reduce} = \log_2 k \cdot (T_{\oplus} + 2 \cdot T_{comm}(m))$$

- worst-case: send and receive are not overlapped in each round

- Choosing the value of k**

- It is the minimum of the function $T_c^{reduce,k}(n)$ over k

input collection (balanced tree)

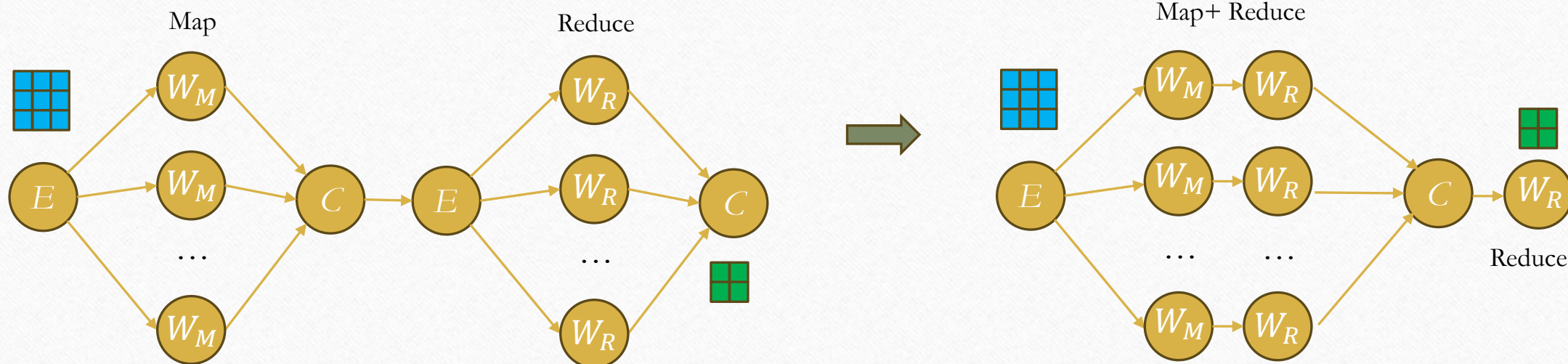


Map+Reduce

1/2

- The Reduce operation is often combined with the Map (or other data parallel patterns, e.g., [stencil](#))
- Logically, this is a pipeline of the two stages, one computing the Map, and one computing the Reduce
- If we have a stream of collections and no dependencies, the two phases can be actually executed in [pipeline](#) :

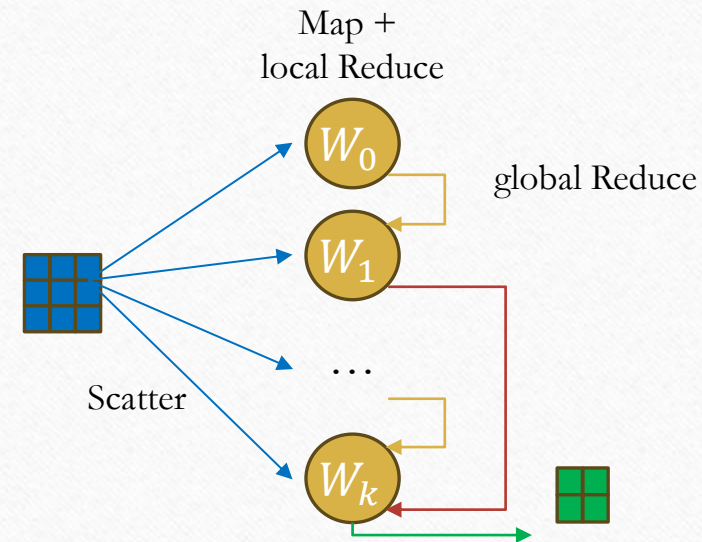
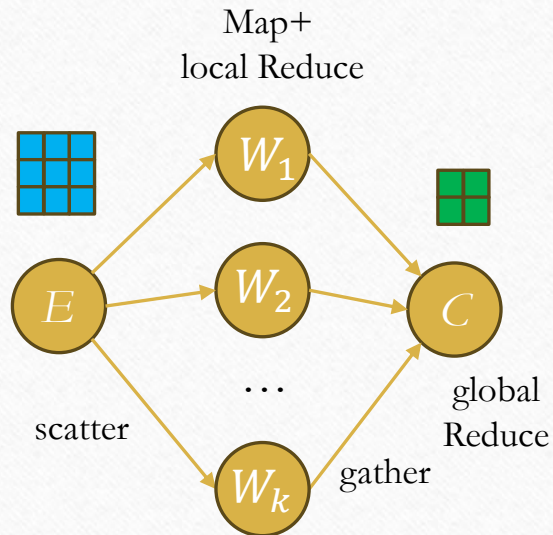
$\text{pipe}(\text{Map}(F), \text{Reduce}(\oplus))$



Map+Reduce

2/2

- However, in practice, a common implementation skeleton combines Map and Reduce operations
 - **Scatter the collections**: each Worker receives a partition of the initial collection
 - **Map + local Reduce**: each Worker computes the Map on the local partition and applies the reduction operator on the local partition as well
 - **Global Reduce**: all Workers exchange (or collect) the result of the local reduce to compute the final value



Example: Map+Reduce on stream

1/2

- Let's consider a stream of n input collections $(C_1, \dots, C_n)$ and the following sequential program

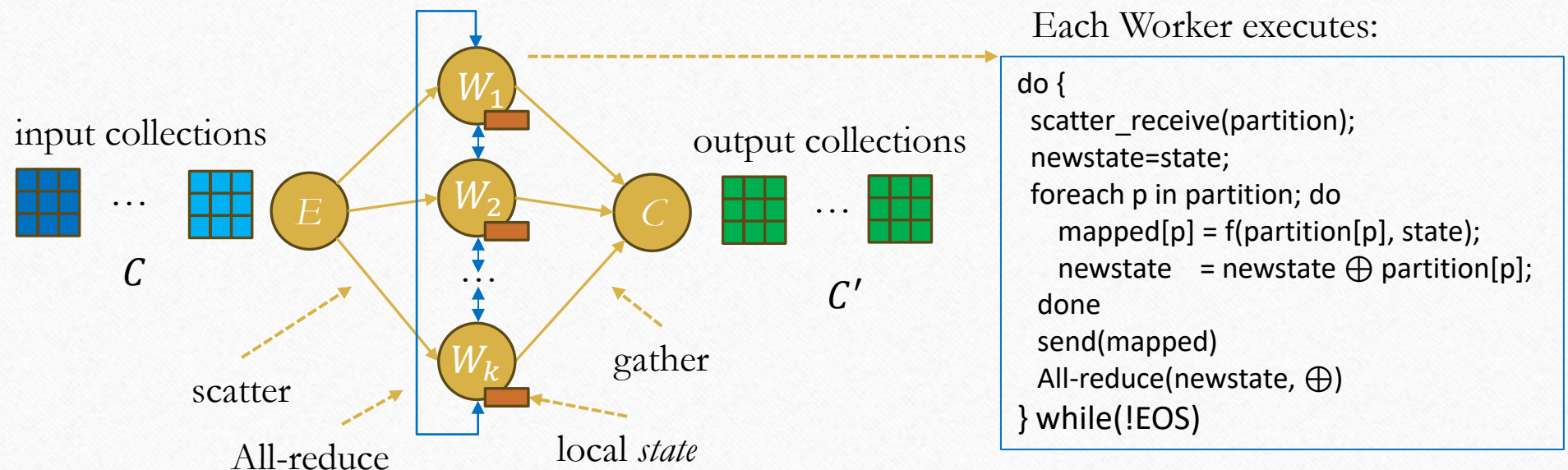
```
state = <initial-value>
for c in  $\{C_1, \dots, C_n\}$  {
  mapped = Map(c, f, state);
  state = Reduce(mapped,  $\oplus$ )
}
```

- where $\text{Map}(c, f, \text{state})$ computes the function f on each element of the current collection c using the current state
 $\text{mapped}_i = f(c_i, \text{state})$ for $i = 1, \dots, |c|$
 - $\text{Reduce}(\text{mapped}, \oplus)$ updates the state executing a reduce operation on the mapped collection using the $\oplus$ operator
- The computation is **stateful**: to compute the collection C_i we need the state computed at the step $i-1$
 - The Map and Reduce steps cannot be executed in pipeline because of the data dependency on the state (**read-after-write dependency**)

Example: Map+Reduce on stream

2/2

- We can parallelize it in a single step by computing a Map and a local reduction in one step, and then, **synchronously**, compute a **global reduction** to update the state using an **all-reduce**



- The **all-reduce** introduces an **implicit barrier**, therefore, the next iteration starts when the previous one has finished

Suggested Readings

- Prof. Danelutto course note:
 - `notes2024spm-Danelutto.pdf`
 - `Chap10StatePatterns.pdf`